Dijkstra's Algorithm Explanation

Dijkstra's Algorithm is a popular algorithm used to find the shortest path from a source node to all other nodes in a weighted graph. It operates under the following principles:

- **Greedy Approach**: At each step, it selects the node with the smallest known distance from the source and explores its neighbors.
- **Non-negative Weights**: The algorithm assumes that all edge weights are non-negative. If there are negative weights, the algorithm may not yield correct results.
- **Priority Queue**: It uses a priority queue to efficiently fetch the next node with the smallest distance.

Steps of Dijkstra's Algorithm

1. **Initialization**:
    - Create a distance array to store the shortest distance from the source to each vertex. Initialize all distances to infinity, except for the source vertex, which is set to 0.
    - Create a visited array to track which vertices have been processed.
2. **Processing Nodes**:
    - While there are unvisited nodes:
        - Select the unvisited node with the smallest distance.
        - Mark it as visited.
        - Update the distances of its neighbors. If the distance to a neighbor can be shortened by taking the current node, update the neighbor's distance.
3. **Termination**:
    - The algorithm terminates when all nodes have been visited or the smallest distance among the unvisited nodes is infinity (indicating that the remaining nodes are unreachable).

Example Implementation in Java

Here’s a complete Java implementation of Dijkstra's algorithm using an adjacency list representation:

```
java
VerifyOpen In EditorRunCopy code
1import java.util.*;
2
3public class DijkstraExample {
4
5    // Edge class to represent a connection between nodes
6    static class Edge {
7        int target;
8        int weight;
9
10        Edge(int target, int weight) {
11            this.target = target;
12            this.weight = weight;
13        }
14    }
15
16    // Pair class to store vertex and its distance
```

```
    static class Pair implements Comparable<Pair> {
        int vertex;
        int distance;

        Pair(int vertex, int distance) {
            this.vertex = vertex;
            this.distance = distance;
        }

        @Override
        public int compareTo(Pair other) {
            return Integer.compare(this.distance, other.distance);
        }
    }

    public static void dijkstra(List<List<Edge>> graph, int source) {
        int n = graph.size();
        int[] distances = new int[n];
        boolean[] visited = new boolean[n];
        Arrays.fill(distances, Integer.MAX_VALUE);
        distances[source] = 0;

        PriorityQueue<Pair> priorityQueue = new PriorityQueue<>();
        priorityQueue.add(new Pair(source, 0));

        while (!priorityQueue.isEmpty()) {
            Pair current = priorityQueue.poll();
            int currentVertex = current.vertex;

            if (visited[currentVertex]) continue;
            visited[currentVertex] = true;

            // Explore neighbors
            for (Edge edge : graph.get(currentVertex)) {
                if (!visited[edge.target]) {
                    int newDist = distances[currentVertex] + edge.weight;
                    if (newDist < distances[edge.target]) {
                        distances[edge.target] = newDist;
                        priorityQueue.add(new Pair(edge.target, newDist));
                    }
                }
            }
        }

        // Print the shortest distances
        for (int i = 0; i < distances.length; i++) {
            System.out.println("Vertex " + i + " has distance: " +
distances[i]);
        }
    }

    public static void main(String[] args) {
        // Create a graph using adjacency list
        List<List<Edge>> graph = new ArrayList<>();
        for (int i = 0; i < 7; i++) {
            graph.add(new ArrayList<>());
        }

        // Adding edges (undirected graph)
        graph.get(0).add(new Edge(1, 14));
        graph.get(0).add(new Edge(3, 5));
        graph.get(1).add(new Edge(0, 14));
        graph.get(1).add(new Edge(4, 2));
        graph.get(1).add(new Edge(2, 6));
```

```
        graph.get(2).add(new Edge(1, 6));
        graph.get(2).add(new Edge(5, 1));
        graph.get(2).add(new Edge(6, 7));
        graph.get(3).add(new Edge(0, 5));
        graph.get(3).add(new Edge(4, 3));
        graph.get(4).add(new Edge(3, 3));
        graph.get(4).add(new Edge(1, 2));
        graph.get(5).add(new Edge(2, 1));
        graph.get(5).add(new Edge(6, 8));
        graph.get(6).add(new Edge(2, 7));
        graph.get(6).add(new Edge(5, 8));

        // Run Dijkstra's algorithm from source vertex 0
        dijkstra(graph, 0);
    }
}
```

Explanation of the Code

- **Graph Representation**: The graph is represented as a list of lists, where each list contains edges (target vertex and weight).
- **Dijkstra's Function**: The `dijkstra` function implements the algorithm, updating distances and using a priority queue to manage the vertices.
- **Output**: The program prints the shortest distance from the source vertex to all other vertices.

Conclusion

Dijkstra's algorithm is a powerful tool for finding the shortest paths in graphs with non-negative weights. It is widely used in various applications, including routing and navigation systems. The provided Java implementation demonstrates how to effectively apply this algorithm using an adjacency list and a priority queue.

Floyd-Warshall Algorithm Explanation

The **Floyd-Warshall algorithm** is a classic algorithm used to find the shortest paths between all pairs of vertices in a weighted graph. It is particularly useful for dense graphs and works well with both directed and undirected graphs, provided there are no negative cycles.

Key Features:

- **Dynamic Programming Approach**: The algorithm uses a dynamic programming technique to build up solutions to subproblems.
- **Time Complexity**: $O(n^3)$, where n is the number of vertices.
- **Space Complexity**: $O(n^2)$ due to the storage of the distance matrix.

How It Works:

1. **Initialization**: Create a distance matrix `D` where `D[i][j]` is initialized to the weight of the edge from vertex `i` to vertex `j`. If there is no edge, set it to infinity (∞). The diagonal elements `D[i][i]` are set to 0.
2. **Iterative Updates**: For each vertex `k`, update the distance matrix:

- For each pair of vertices `(i, j)`, check if the path from `i` to `j` through `k` is shorter than the current known distance. If it is, update `D[i][j]`.

3. **Final Matrix**: After processing all vertices, the matrix `D` will contain the shortest distances between every pair of vertices.

Example in Java

Here is a complete implementation of the Floyd-Warshall algorithm in Java:

```
java
VerifyOpen In EditorRunCopy code
1import java.util.Arrays;
2
3public class FloydWarshall {
4    final static int INF = 99999; // A large value representing infinity
5    final static int nV = 4; // Number of vertices
6
7    // Implementing Floyd-Warshall algorithm
8    void floydWarshall(int graph[][]) {
9        int[][] distance = new int[nV][nV];
10
11        // Initialize the distance matrix
12        for (int i = 0; i < nV; i++) {
13            for (int j = 0; j < nV; j++) {
14                distance[i][j] = graph[i][j];
15            }
16        }
17
18        // Main algorithm
19        for (int k = 0; k < nV; k++) {
20            for (int i = 0; i < nV; i++) {
21                for (int j = 0; j < nV; j++) {
22                    if (distance[i][k] + distance[k][j] < distance[i][j]) {
23                        distance[i][j] = distance[i][k] + distance[k][j];
24                    }
25                }
26            }
27        }
28
29        // Print the final distance matrix
30        printMatrix(distance);
31    }
32
33    // Function to print the distance matrix
34    void printMatrix(int matrix[][]) {
35        for (int i = 0; i < nV; i++) {
36            for (int j = 0; j < nV; j++) {
37                if (matrix[i][j] == INF) {
38                    System.out.print("INF ");
39                } else {
40                    System.out.print(matrix[i][j] + " ");
41                }
42            }
43            System.out.println();
44        }
45    }
46
47    public static void main(String[] args) {
48        int graph[][] = {
49            {0, 3, INF, 5},
50            {2, 0, INF, 4},
51            {INF, 1, 0, INF},
```

```
52            {INF, INF, 2, 0}
53        };
54        FloydWarshall fw = new FloydWarshall();
55        fw.floydWarshall(graph);
56    }
57}
```

Explanation of the Code:

- **Initialization**: The `graph` matrix represents the weights of the edges. If there is no edge between two vertices, it is represented by `INF`.
- **Distance Matrix Update**: The nested loops iterate through each vertex as an intermediate point and update the shortest paths accordingly.
- **Output**: The final distance matrix is printed, showing the shortest paths between all pairs of vertices.

Conclusion

The Floyd-Warshall algorithm is a powerful tool for finding shortest paths in graphs, especially when you need to consider all pairs of vertices. Its straightforward implementation and ability to handle negative weights (as long as there are no negative cycles) make it a valuable algorithm in graph theory.

Prim's Algorithm Explanation

Prim's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, undirected graph. The MST is a subset of the edges that connects all the vertices together without any cycles and with the minimum possible total edge weight.

How Prim's Algorithm Works

1. **Initialization**:
    - Start with a vertex chosen at random and add it to the MST.
    - Create a set to keep track of the vertices included in the MST.
2. **Edge Selection**:
    - Find all the edges that connect the vertices in the MST to the vertices not yet included.
    - Select the edge with the smallest weight and add it to the MST.
3. **Repeat**:
    - Continue the process until all vertices are included in the MST.

Key Characteristics

- **Time Complexity**: The time complexity of Prim's algorithm is (O(V^2)) when using an adjacency matrix, where (V) is the number of vertices.
- **Applications**: It is used in network design, such as laying cables or designing network protocols.

Example of Prim's Algorithm in Java

Here is a detailed implementation of Prim's algorithm in Java using an adjacency matrix:

```
java
VerifyOpen In EditorRunCopy code
1import java.util.Arrays;
2
3class PGraph {
4    public void Prim(int G[][], int V) {
5        int INF = 9999999; // A large value representing infinity
6        int no_edge; // Number of edges in the MST
7
8        // Array to track selected vertices
9        boolean[] selected = new boolean[V];
10        Arrays.fill(selected, false); // Initially, no vertices are selected
11
12        no_edge = 0; // Initialize number of edges to 0
13        selected[0] = true; // Start with the first vertex
14
15        System.out.println("Edge : Weight");
16
17        while (no_edge < V - 1) {
18            int min = INF; // Initialize minimum weight
19            int x = 0; // Row number
20            int y = 0; // Column number
21
22            // Iterate through the selected vertices
23            for (int i = 0; i < V; i++) {
24                if (selected[i]) {
25                    // Check all adjacent vertices
26                    for (int j = 0; j < V; j++) {
27                        // If the vertex is not selected and there is an edge
28                        if (!selected[j] && G[i][j] != 0) {
29                            // Update minimum weight and edge
30                            if (min > G[i][j]) {
31                                min = G[i][j];
32                                x = i;
33                                y = j;
34                            }
35                        }
36                    }
37                }
38            }
39            System.out.println(x + " - " + y + " : " + G[x][y]); // Print the
selected edge
40            selected[y] = true; // Mark the vertex as selected
41            no_edge++; // Increment the number of edges
42        }
43    }
44
45    public static void main(String[] args) {
46        PGraph g = new PGraph();
47        int V = 5; // Number of vertices
48        int[][] G = {
49            {0, 9, 75, 0, 0},
50            {9, 0, 95, 19, 42},
51            {75, 95, 0, 51, 66},
52            {0, 19, 51, 0, 31},
53            {0, 42, 66, 31, 0}
54        };
55
56        g.Prim(G, V); // Call the Prim's algorithm method
57    }
```

```
58}
```

Explanation of the Code

- **Graph Representation**: The graph is represented using an adjacency matrix `G`, where `G[i][j]` holds the weight of the edge between vertex `i` and vertex `j`.
- **Selected Array**: The `selected` array keeps track of which vertices are included in the MST.
- **Main Loop**: The main loop continues until we have included `V-1` edges in the MST. It finds the minimum weight edge connecting the selected vertices to the unselected vertices and adds it to the MST.

This implementation provides a clear and efficient way to understand and apply Prim's algorithm in Java.

Kruskal's Algorithm Explanation

Kruskal's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, undirected graph. The MST is a subset of the edges that connects all vertices without any cycles and with the minimum possible total edge weight. Here’s how the algorithm works:

Steps of Kruskal's Algorithm

1. **Sort All Edges**:
    - Begin by sorting all the edges in the graph in non-decreasing order of their weights.
2. **Initialize Subsets**:
    - Create a disjoint set (or union-find) for each vertex to keep track of which vertices are in which components.
3. **Iterate Over Sorted Edges**:
    - For each edge in the sorted list, check if adding it to the MST would form a cycle.
    - If it does not form a cycle, include it in the MST and union the two vertices.
4. **Repeat Until MST is Complete**:
    - Continue this process until you have included (V-1) edges in the MST, where (V) is the number of vertices in the graph.

Key Concepts

- **Union-Find Data Structure**: This is used to efficiently manage and merge disjoint sets, allowing the algorithm to check for cycles quickly.
- **Greedy Approach**: The algorithm always picks the smallest edge available that does not form a cycle, ensuring that the total weight of the MST is minimized.

Example Implementation in Java

Here’s a complete Java implementation of Kruskal's algorithm:

```
java
VerifyOpen In EditorRunCopy code
```

```
import java.util.Arrays;

class Edge implements Comparable<Edge> {
    int src, dest, weight;

    public int compareTo(Edge compareEdge) {
        return this.weight - compareEdge.weight;
    }
}

class Subset {
    int parent, rank;
}

class Graph {
    int V, E; // Number of vertices and edges
    Edge[] edges; // Collection of all edges

    Graph(int v, int e) {
        V = v;
        E = e;
        edges = new Edge[E];
        for (int i = 0; i < e; ++i) {
            edges[i] = new Edge();
        }
    }

    // A utility function to find the set of an element i (uses path
compression)
    int find(Subset[] subsets, int i) {
        if (subsets[i].parent != i)
            subsets[i].parent = find(subsets, subsets[i].parent);
        return subsets[i].parent;
    }

    // A function that does union of two sets of x and y (uses union by rank)
    void union(Subset[] subsets, int x, int y) {
        int xroot = find(subsets, x);
        int yroot = find(subsets, y);

        if (subsets[xroot].rank < subsets[yroot].rank)
            subsets[xroot].parent = yroot;
        else if (subsets[xroot].rank > subsets[yroot].rank)
            subsets[yroot].parent = xroot;
        else {
            subsets[yroot].parent = xroot;
            subsets[xroot].rank++;
        }
    }

    // The main function to construct MST using Kruskal's algorithm
    void kruskalMST() {
        Edge[] result = new Edge[V]; // This will store the resultant MST
        int e = 0; // An index variable, used for result[]
        int i = 0; // An index variable, used for sorted edges
        for (i = 0; i < V; ++i)
            result[i] = new Edge();

        // Step 1: Sort all the edges in non-decreasing order of their weight.
        Arrays.sort(edges);

        // Allocate memory for creating V subsets
        Subset[] subsets = new Subset[V];
        for (i = 0; i < V; ++i) {
```

```
64            subsets[i] = new Subset();
65            subsets[i].parent = i;
66            subsets[i].rank = 0;
67        }
68
69        i = 0; // Index used to pick the smallest edge
70
71        // Number of edges to be taken is equal to V-1
72        while (e < V - 1) {
73            // Step 2: Pick the smallest edge. And increment the index for
next iteration
74            Edge next_edge = edges[i++];
75
76            int x = find(subsets, next_edge.src);
77            int y = find(subsets, next_edge.dest);
78
79            // If including this edge does not cause a cycle
80            if (x != y) {
81                result[e++] = next_edge;
82                union(subsets, x, y);
83            }
84        }
85
86        // Print the contents of result[] to display the built MST
87        System.out.println("Following are the edges in the constructed MST:");
88        for (i = 0; i < e; ++i)
89            System.out.println(result[i].src + " -- " + result[i].dest + " ==
" + result[i].weight);
90    }
91
92    public static void main(String[] args) {
93        int V = 4; // Number of vertices in graph
94        int E = 5; // Number of edges in graph
95        Graph graph = new Graph(V, E);
96
97        // Add edges
98        graph.edges[0].src = 0;
99        graph.edges[0].dest = 1;
100        graph.edges[0].weight = 10;
101
102        graph.edges[1].src = 0;
103        graph.edges[1].dest = 2;
104        graph.edges[1].weight = 6;
105
106        graph.edges[2].src = 0;
107        graph.edges[2].dest = 3;
108        graph.edges[2].weight = 5;
109
110        graph.edges[3].src = 1;
111        graph.edges[3].dest = 3;
112        graph.edges[3].weight = 15;
113
114        graph.edges[4].src = 2;
115        graph.edges[4].dest = 3;
116        graph.edges[4].weight = 4;
117
118        // Function call
119        graph.kruskalMST();
120    }
121}
```

Explanation of the Code

- **Edge Class**: Represents an edge with source, destination, and weight. It implements `Comparable` to sort edges by weight.
- **Subset Class**: Represents a subset for union-find operations, containing the parent and rank.
- **Graph Class**: Contains methods for finding the MST using Kruskal's algorithm.
    - `find()`: Uses path compression to find the root of the set.
    - `union()`: Merges two sets using union by rank.
    - `kruskalMST()`: Main method to perform Kruskal's algorithm and print the MST.

Conclusion

Kruskal's algorithm is efficient for finding the minimum spanning tree of a graph, especially when the graph is sparse. It is widely used in network design, clustering, and various optimization problems. The provided Java implementation demonstrates how to apply the algorithm effectively.

Final Answer

Shortest Path Algorithms

Dijkstra's Algorithm Explanation

Dijkstra's Algorithm is a widely used algorithm for finding the shortest path from a source node to all other nodes in a weighted graph.

- **Key Characteristics**:
    - **Greedy Approach**: Selects the node with the smallest known distance at each step.
    - **Non-negative Weights**: Assumes all edge weights are non-negative.
    - **Priority Queue**: Utilizes a priority queue for efficient retrieval of the next node.

Steps of Dijkstra's Algorithm

1. **Initialization**:
    - Create a distance array initialized to infinity, except for the source node (set to 0).
    - Create a visited array to track processed nodes.
2. **Processing Nodes**:
    - While there are unvisited nodes:
        - Select the unvisited node with the smallest distance.
        - Mark it as visited.
        - Update distances of its neighbors.
3. **Termination**:
    - The algorithm ends when all nodes are visited or the smallest distance among unvisited nodes is infinity.

Example Implementation in Java

```
java
VerifyOpen In EditorRunCopy code
1import java.util.*;
```

```
2
3public class DijkstraExample {
4    static class Edge {
5        int target, weight;
6        Edge(int target, int weight) {
7            this.target = target;
8            this.weight = weight;
9        }
10    }
11
12    static class Pair implements Comparable<Pair> {
13        int vertex, distance;
14        Pair(int vertex, int distance) {
15            this.vertex = vertex;
16            this.distance = distance;
17        }
18        @Override
19        public int compareTo(Pair other) {
20            return Integer.compare(this.distance, other.distance);
21        }
22    }
23
24    public static void dijkstra(List<List<Edge>> graph, int source) {
25        int n = graph.size();
26        int[] distances = new int[n];
27        boolean[] visited = new boolean[n];
28        Arrays.fill(distances, Integer.MAX_VALUE);
29        distances[source] = 0;
30
31        PriorityQueue<Pair> priorityQueue = new PriorityQueue<>();
32        priorityQueue.add(new Pair(source, 0));
33
34        while (!priorityQueue.isEmpty()) {
35            Pair current = priorityQueue.poll();
36            int currentVertex = current.vertex;
37
38            if (visited[currentVertex]) continue;
39            visited[currentVertex] = true;
40
41            for (Edge edge : graph.get(currentVertex)) {
42                if (!visited[edge.target]) {
43                    int newDist = distances[currentVertex] + edge.weight;
44                    if (newDist < distances[edge.target]) {
45                        distances[edge.target] = newDist;
46                        priorityQueue.add(new Pair(edge.target, newDist));
47                    }
48                }
49            }
50        }
51
52        for (int i = 0; i < distances.length; i++) {
53            System.out.println("Vertex " + i + " has distance: " +
distances[i]);
54        }
55    }
56
57    public static void main(String[] args) {
58        List<List<Edge>> graph = new ArrayList<>();
59        for (int i = 0; i < 7; i++) {
60            graph.add(new ArrayList<>());
61        }
62        // Adding edges (undirected graph)
63        graph.get(0).add(new Edge(1, 14));
64        graph.get(0).add(new Edge(3, 5));
```

```
65        graph.get(1).add(new Edge(0, 14));
66        graph.get(1).add(new Edge(4, 2));
67        graph.get(1).add(new Edge(2, 6));
68        graph.get(2).add(new Edge(1, 6));
69        graph.get(2).add(new Edge(5, 1));
70        graph.get(2).add(new Edge(6, 7));
71        graph.get(3).add(new Edge(0, 5));
72        graph.get(3).add(new Edge(4, 3));
73        graph.get(4).add(new Edge(3, 3));
74        graph.get(4).add(new Edge(1, 2));
75        graph.get(5).add(new Edge(2, 1));
76        graph.get(5).add(new Edge(6, 8));
77        graph.get(6).add(new Edge(2, 7));
78        graph.get(6).add(new Edge(5, 8));
79
80        dijkstra(graph, 0);
81    }
82}
```

Floyd-Warshall Algorithm Explanation

The **Floyd-Warshall algorithm** finds the shortest paths between all pairs of vertices in a weighted graph.

Key Features:

- **Dynamic Programming Approach**: Builds solutions to subproblems.
- **Time Complexity**: $O(n^3)$, where n is the number of vertices.
- **Space Complexity**: $O(n^2)$ for the distance matrix.

Steps of Floyd-Warshall Algorithm

1. **Initialization**: Create a distance matrix initialized to edge weights or infinity.
2. **Iterative Updates**: For each vertex `k`, update the distance matrix for all pairs `(i, j)`.
3. **Final Matrix**: The matrix will contain the shortest distances after processing.

Example Implementation in Java

```
java
VerifyOpen In EditorRunCopy code
1import java.util.Arrays;
2
3public class FloydWarshall {
4    final static int INF = 99999; // A large value representing infinity
5    final static int nV = 4; // Number of vertices
6
7    void floydWarshall(int graph[][]) {
8        int[][] distance = new int[nV][nV];
9
10       for (int i = 0; i < nV; i++) {
11           for (int j = 0; j < nV; j++) {
12               distance[i][j] = graph[i][j];
13           }
14       }
15
16       for (int k = 0; k < nV; k++) {
17           for (int i = 0; i < nV; i++) {
18               for (int j = 0; j < nV; j++) {
19                   if (distance[i][k] + distance[k][j] < distance[i][j]) {
20                       distance[i][j] = distance[i][k] + distance[k][j];
```

```
                    }
                }
            }
        }

        printMatrix(distance);
    }

    void printMatrix(int matrix[][]) {
        for (int i = 0; i < nV; i++) {
            for (int j = 0; j < nV; j++) {
                if (matrix[i][j] == INF) {
                    System.out.print("INF ");
                } else {
                    System.out.print(matrix[i][j] + " ");
                }
            }
            System.out.println();
        }
    }

    public static void main(String[] args) {
        int graph[][] = {
            {0, 3, INF, 5},
            {2, 0, INF, 4},
            {INF, 1, 0, INF},
            {INF, INF, 2, 0}
        };
        FloydWarshall fw = new FloydWarshall();
        fw.floydWarshall(graph);
    }
}
```

Spanning Trees

Prim's Algorithm Explanation

Prim's Algorithm is a greedy algorithm for finding the Minimum Spanning Tree (MST) of a connected, undirected graph.

Steps of Prim's Algorithm

1. **Initialization**: Start with a random vertex and add it to the MST.
2. **Edge Selection**: Find the smallest edge connecting the MST to a vertex not in the MST.
3. **Repeat**: Continue until all vertices are included.

Example Implementation in Java

```
java
VerifyOpen In EditorRunCopy code
1import java.util.Arrays;
2
3class PGraph {
4    public void Prim(int G[][], int V) {
5        int INF = 9999999; // A large value representing infinity
6        boolean[] selected = new boolean[V];
7        Arrays.fill(selected, false);
8        selected[0] = true;
9
10        System.out.println("Edge : Weight");
11
```

```
        for (int no_edge = 0; no_edge < V - 1; ) {
            int min = INF, x = 0, y = 0;

            for (int i = 0; i < V; i++) {
                if (selected[i]) {
                    for (int j = 0; j < V; j++) {
                        if (!selected[j] && G[i][j] != 0) {
                            if (min > G[i][j]) {
                                min = G[i][j];
                                x = i;
                                y = j;
                            }
                        }
                    }
                }
            }
            System.out.println(x + " - " + y + " : " + G[x][y]);
            selected[y] = true;
            no_edge++;
        }
    }

    public static void main(String[] args) {
        PGraph g = new PGraph();
        int V = 5; // Number of vertices
        int[][] G = {
            {0, 9, 75, 0, 0},
            {9, 0, 95, 19, 42},
            {75, 95, 0, 51, 66},
            {0, 19, 51, 0, 31},
            {0, 42, 66, 31, 0}
        };
        g.Prim(G, V);
    }
}
```

Kruskal's Algorithm Explanation

Kruskal's Algorithm is another greedy algorithm for finding the Minimum Spanning Tree (MST).

Steps of Kruskal's Algorithm

1. **Sort All Edges**: Sort edges in non-decreasing order of weight.
2. **Initialize Subsets**: Create a disjoint set for each vertex.
3. **Iterate Over Sorted Edges**: Include edges that do not form a cycle until (V-1) edges are included.

Example Implementation in Java

```
java
VerifyOpen In EditorRunCopy code
1import java.util.Arrays;
2
3class Edge implements Comparable<Edge> {
4    int src, dest, weight;
5    public int compareTo(Edge compareEdge) {
6        return this.weight - compareEdge.weight;
7    }
8}
9
10class Subset {
```

```
    int parent, rank;
}

class Graph {
    int V, E; // Number of vertices and edges
    Edge[] edges; // Collection of all edges

    Graph(int v, int e) {
        V = v;
        E = e;
        edges = new Edge[E];
        for (int i = 0; i < e; ++i) {
            edges[i] = new Edge();
        }
    }

    int find(Subset[] subsets, int i) {
        if (subsets[i].parent != i)
            subsets[i].parent = find(subsets, subsets[i].parent);
        return subsets[i].parent;
    }

    void union(Subset[] subsets, int x, int y) {
        int xroot = find(subsets, x);
        int yroot = find(subsets, y);
        if (subsets[xroot].rank < subsets[yroot].rank)
            subsets[xroot].parent = yroot;
        else if (subsets[xroot].rank > subsets[yroot].rank)
            subsets[yroot].parent = xroot;
        else {
            subsets[yroot].parent = xroot;
            subsets[xroot].rank++;
        }
    }

    void kruskalMST() {
        Edge[] result = new Edge[V]; // This will store the resultant MST
        int e = 0; // An index variable, used for result[]
        int i = 0; // An index variable, used for sorted edges
        for (i = 0; i < V; ++i)
            result[i] = new Edge();

        Arrays.sort(edges);

        Subset[] subsets = new Subset[V];
        for (i = 0; i < V; ++i) {
            subsets[i] = new Subset();
            subsets[i].parent = i;
            subsets[i].rank = 0;
        }

        i = 0; // Index used to pick the smallest edge

        while (e < V - 1) {
            Edge next_edge = edges[i++];
            int x = find(subsets, next_edge.src);
            int y = find(subsets, next_edge.dest);
            if (x != y) {
                result[e++] = next_edge;
                union(subsets, x, y);
            }
        }

        System.out.println("Following are the edges in the constructed MST:");
```

```
75        for (i = 0; i < e; ++i)
76            System.out.println(result[i].src + " -- " + result[i].dest + " ==
" + result[i].weight);
77    }
78
79    public static void main(String[] args) {
80        int V = 4; // Number of vertices in graph
81        int E = 5; // Number of edges in graph
82        Graph graph = new Graph(V, E);
83        graph.edges[0].src = 0; graph.edges[0].dest = 1; graph.edges[0].weight
= 10;
84        graph.edges[1].src = 0; graph.edges[1].dest = 2; graph.edges[1].weight
= 6;
85        graph.edges[2].src = 0; graph.edges[2].dest = 3; graph.edges[2].weight
= 5;
86        graph.edges[3].src = 1; graph.edges[3].dest = 3; graph.edges[3].weight
= 15;
87        graph.edges[4].src = 2; graph.edges[4].dest = 3; graph.edges[4].weight
= 4;
88
89        graph.kruskalMST();
90    }
91}
```

Conclusion

These algorithms are fundamental in graph theory and have various applications in computer science, including network design, routing, and optimization problems. Understanding their implementations.